Lecture 4

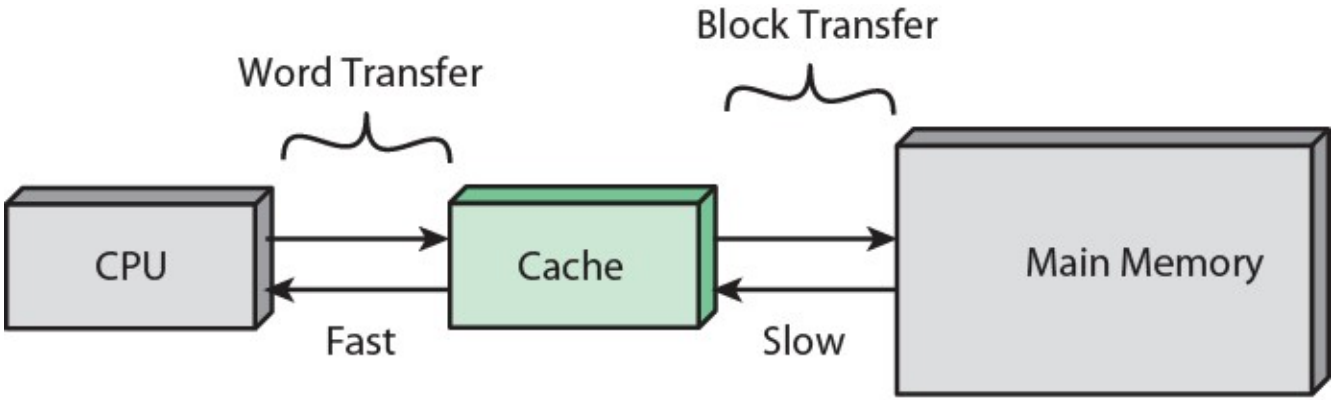
Software Architecture - DSM

Institute of Computer Science and Information Technology,
The University of Agriculture, Peshawar.

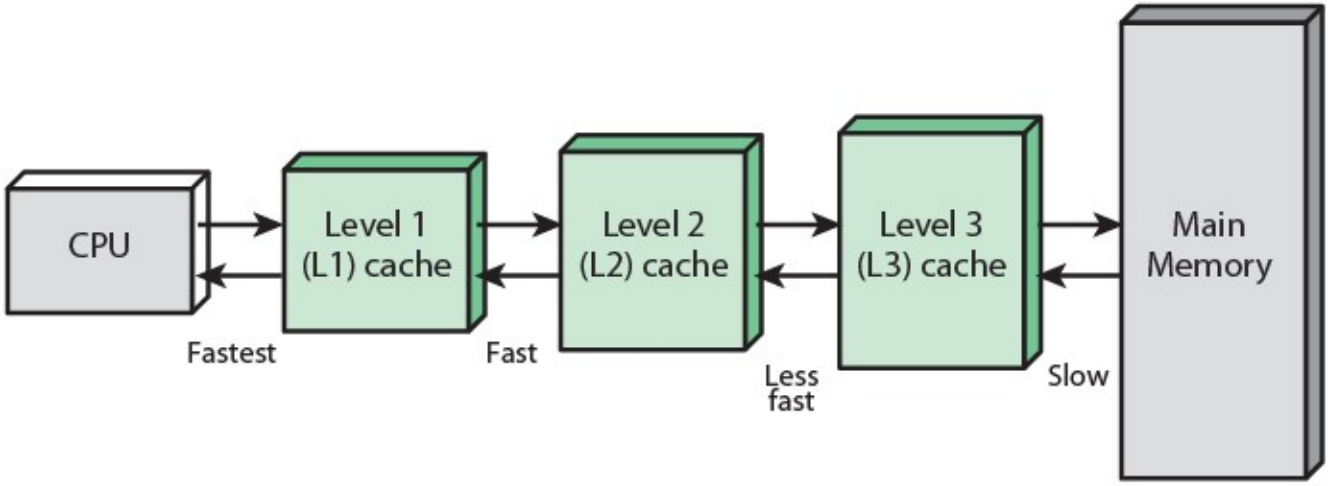
Cache

- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module

Cache and Main Memory

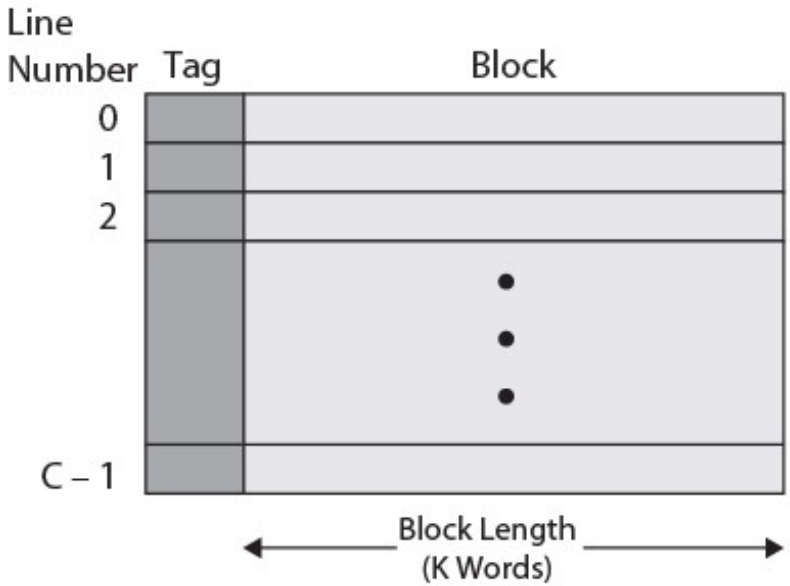


(a) Single cache

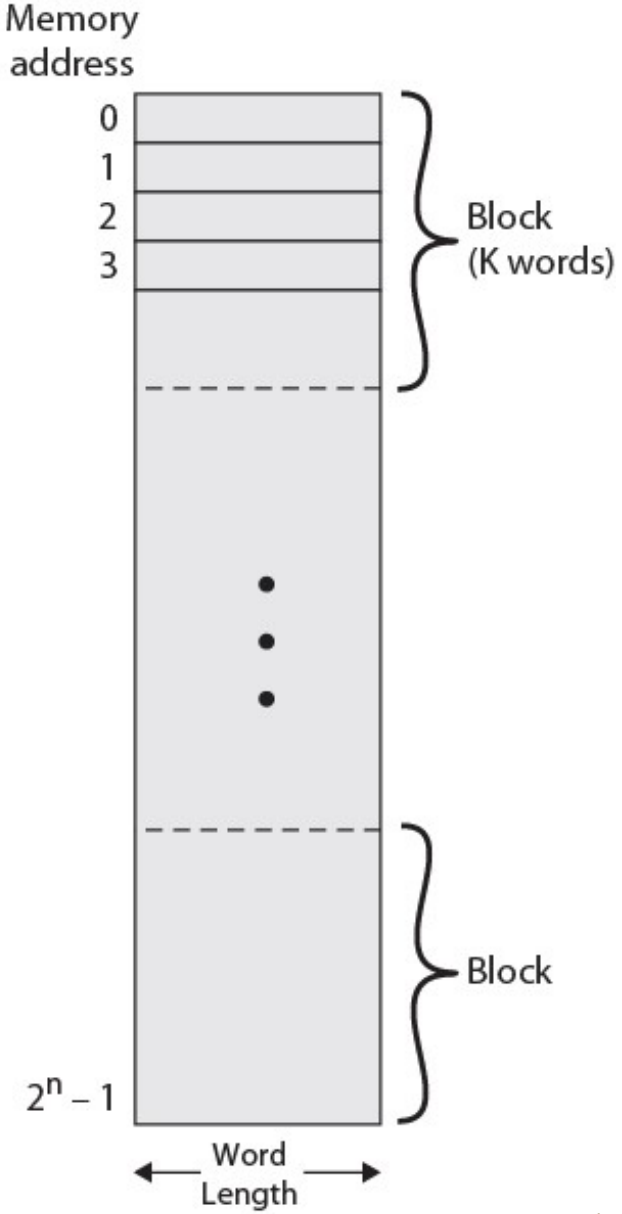


(b) Three-level cache organization

Cache/Main Memory Structure



(a) Cache



(b) Main memory

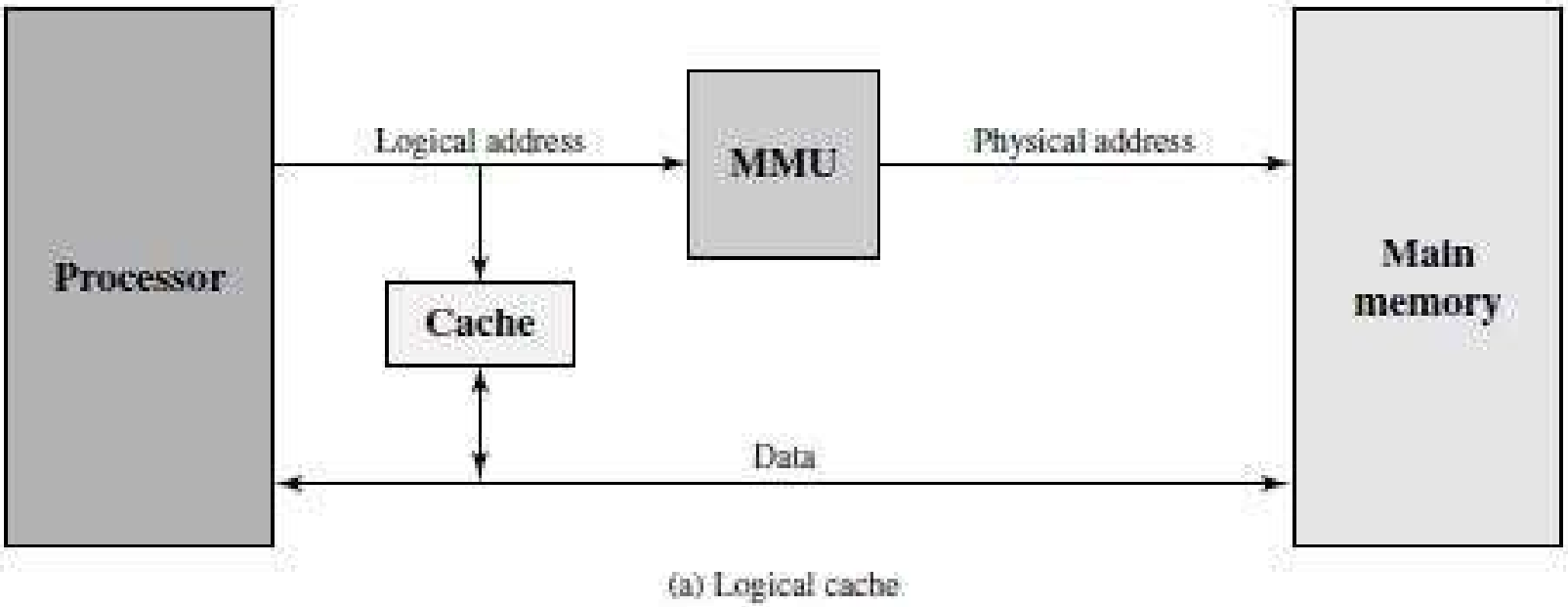
Cache operation – overview

- CPU requests contents of memory location

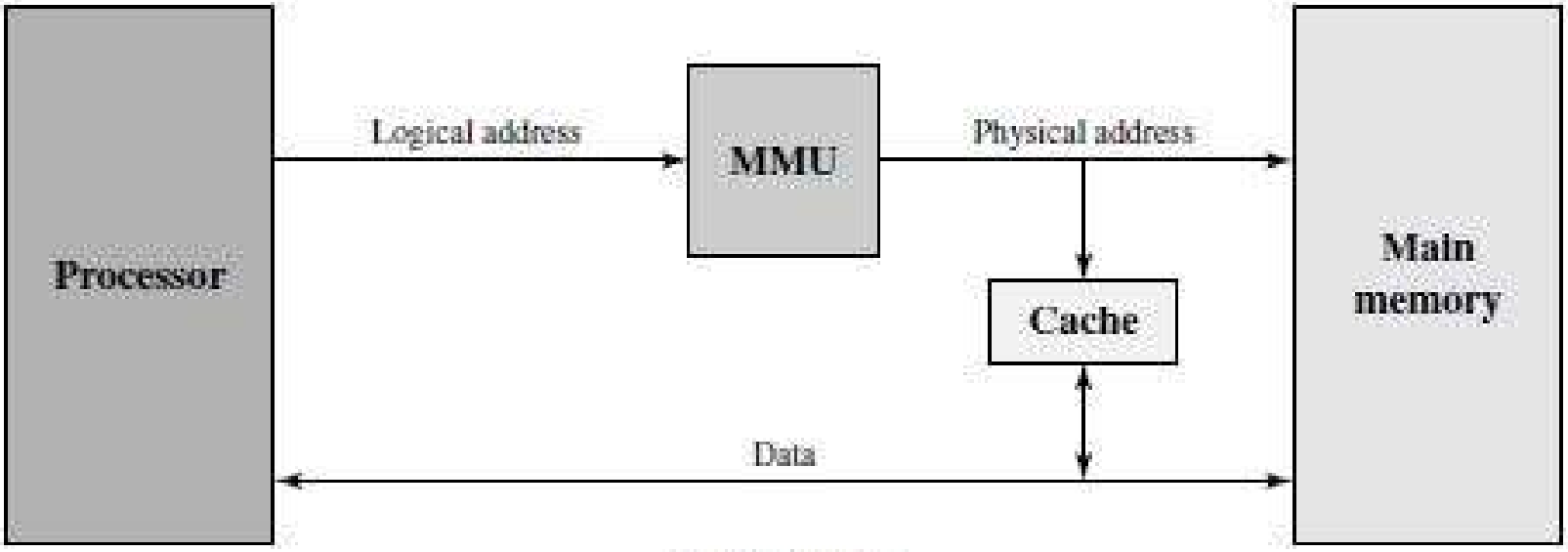
- Check cache for this data
- If present, get from cache (fast)
- If not present, read required block from main memory to cache
- Then deliver from cache to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

Cache Addressing

- Where does cache sit?
  - Between processor and virtual Memory Management Unit (MMU)
  - Between MMU and main memory
- Logical cache (virtual cache) stores data using virtual addresses
  - Processor accesses cache directly, not through physical cache
  - Cache access faster, before MMU address translation
  - Virtual addresses use same address space for different applications
    - Must flush cache on each context switch
- Physical cache stores data using main memory physical addresses



(a) Logical cache



(b) Physical cache

Figure 4.7 Logical and Physical Caches

Virtual Memory

- Virtual memory is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available.
- When virtual memory is used, the address fields of machine instructions contain virtual addresses.
- For reads to and writes from main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory.
- When virtual addresses are used, the system designer may choose to place the cache between the processor and the MMU or between the MMU and main memory.

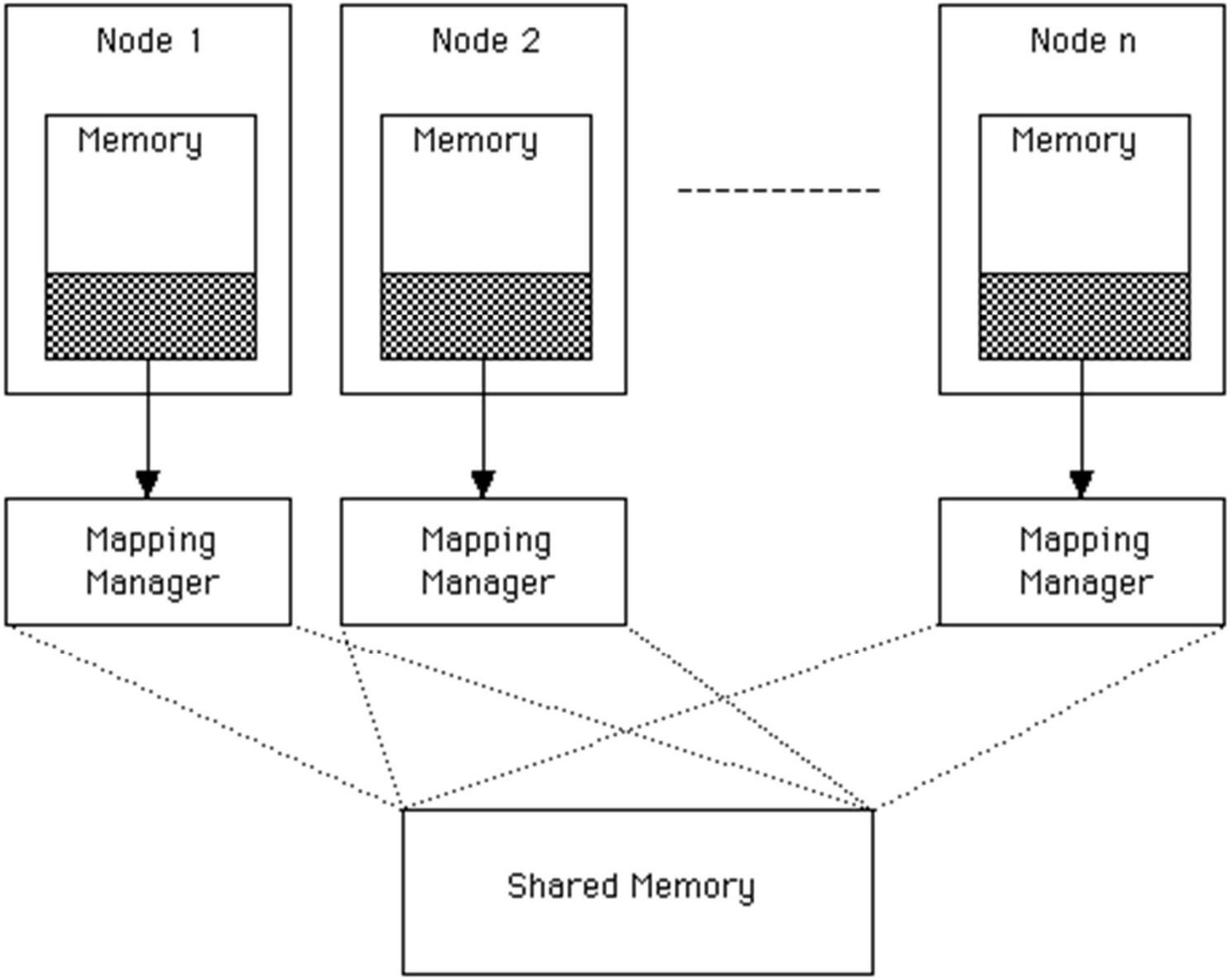
Distributed Shared Memory

- Distributed Shared Memory (DSM) is a form of memory architecture where physically separated memories can be addressed as one logically shared addressed pace.
- As the word "Distributed" is attached with the term "Shared" so it does not mean that there is a single centralized memory, but that the address space is "shared".
- In DSM, data is accessed from a shared address space similar to the way that virtual memory is accessed.
- Information moves between main memory and secondary memory (with in a node) and between main recollections of various nodes.

DSM (1)

- DSM system implements the share memory model on a physically distributed memory system.
- A shared virtual address space is maintained such that the same virtual address across processors refers to the same logical data, regardless of where that data is physically stored.
- When implemented in the OS, such systems are transparent to the developer; whereas the underlying distributed memory is completely hidden from the users.

DSM(2)

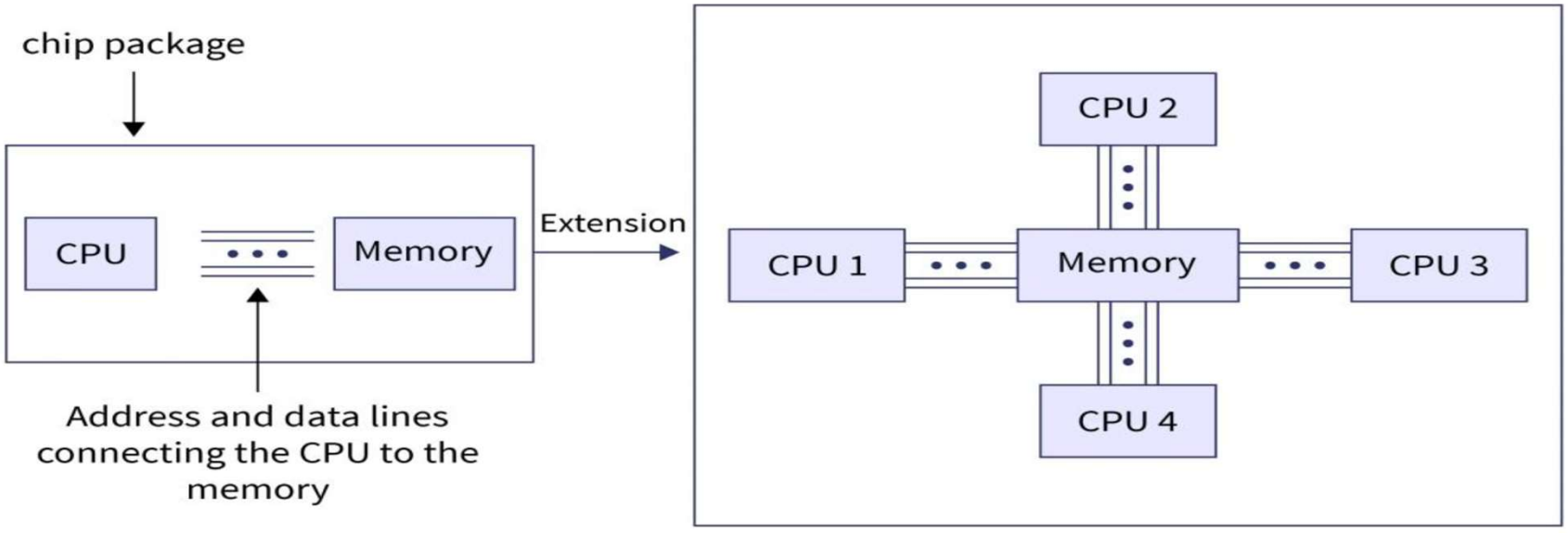

Node 1
Memory
Node 2
Memory
Node n
Memory
Mapping Manager
Mapping Manager
Mapping Manager
Shared Memory

DSM Overview

- **DSM Implementation**
  - Supported in both hardware and software.
- **Hardware-Based DSM**
  - On-Chip Memory
  - Bus-Based Multiprocessors
  - Ring-Based Multiprocessors
- **Software-Based DSM**
  - Implemented at the Operating System (OS) level
  - Implemented using libraries

On-Chip Memory

- The **CPU portion** of the chip stores data.
- The **address lines** are directly connected to the CPU memory, enabling easy access.
- It is **expensive** to implement.
- Its **construction** is complex.



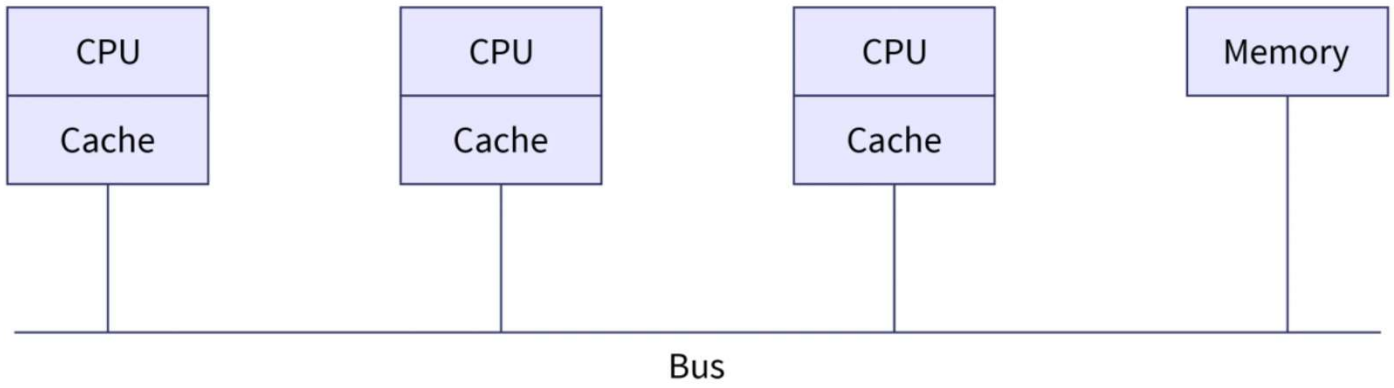
A single-chip computer

A hypothetical shared-memory multiprocessor

Bus-Based Multiprocessors

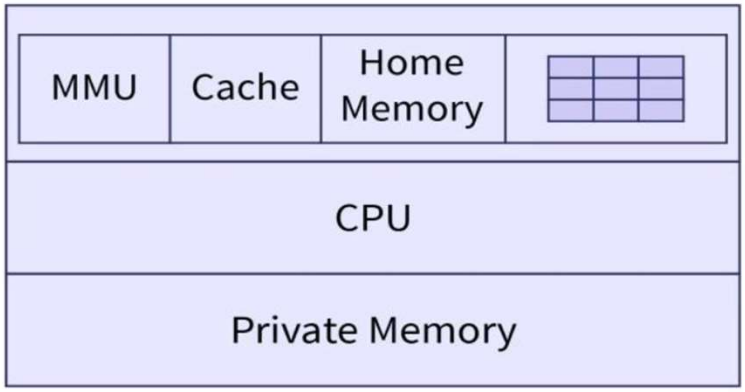
- In this system, a **multiprocessor with caches** is connected to the **main memory** via a **shared bus**.
- Each **processor has its own cache**.
- The **cache helps reduce network traffic**.
- **Prevention algorithms** are required to avoid simultaneous memory access by multiple CPUs.

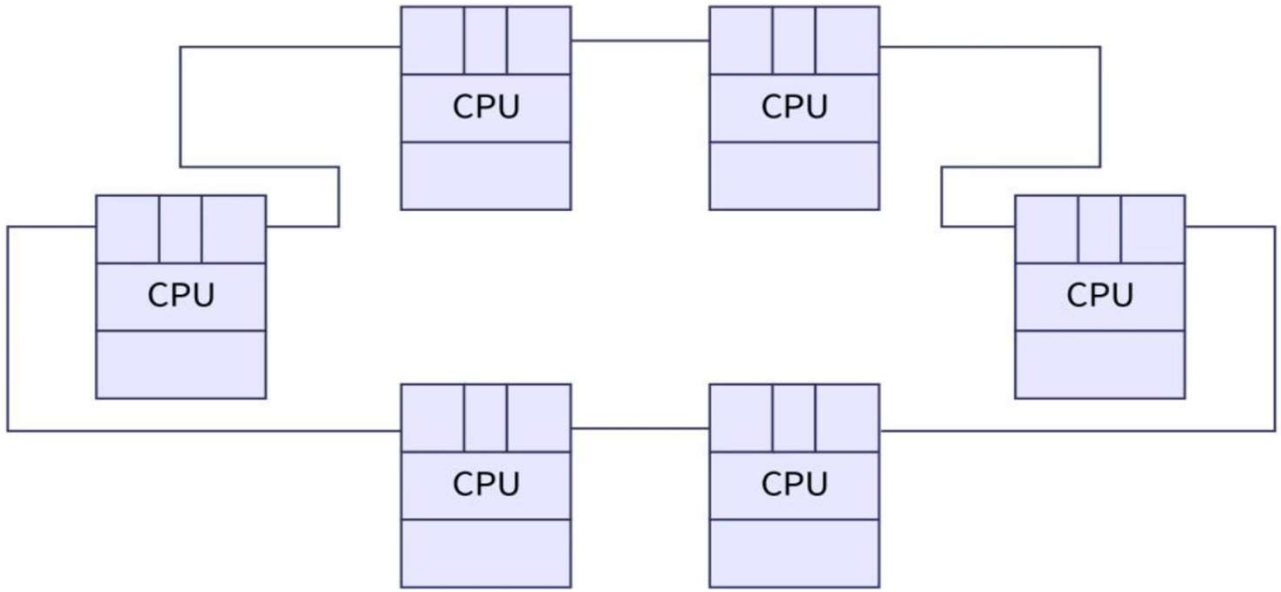
A multiprocessor with caching



Ring-Based Multiprocessors

- **Ring-Based DSM** does not have a **global centralized memory**.
- All **nodes** are connected via a **token-passing ring**.
- In **Ring-Based DSM**, a **single address line** is divided into a **shared area**.
- The **single address space** is split into a **private** and a **shared** section.
- The **private section** is further divided into **regions**, allowing each machine to maintain its **own stack** for unshared data.
- The **shared section** is accessible to all nodes.





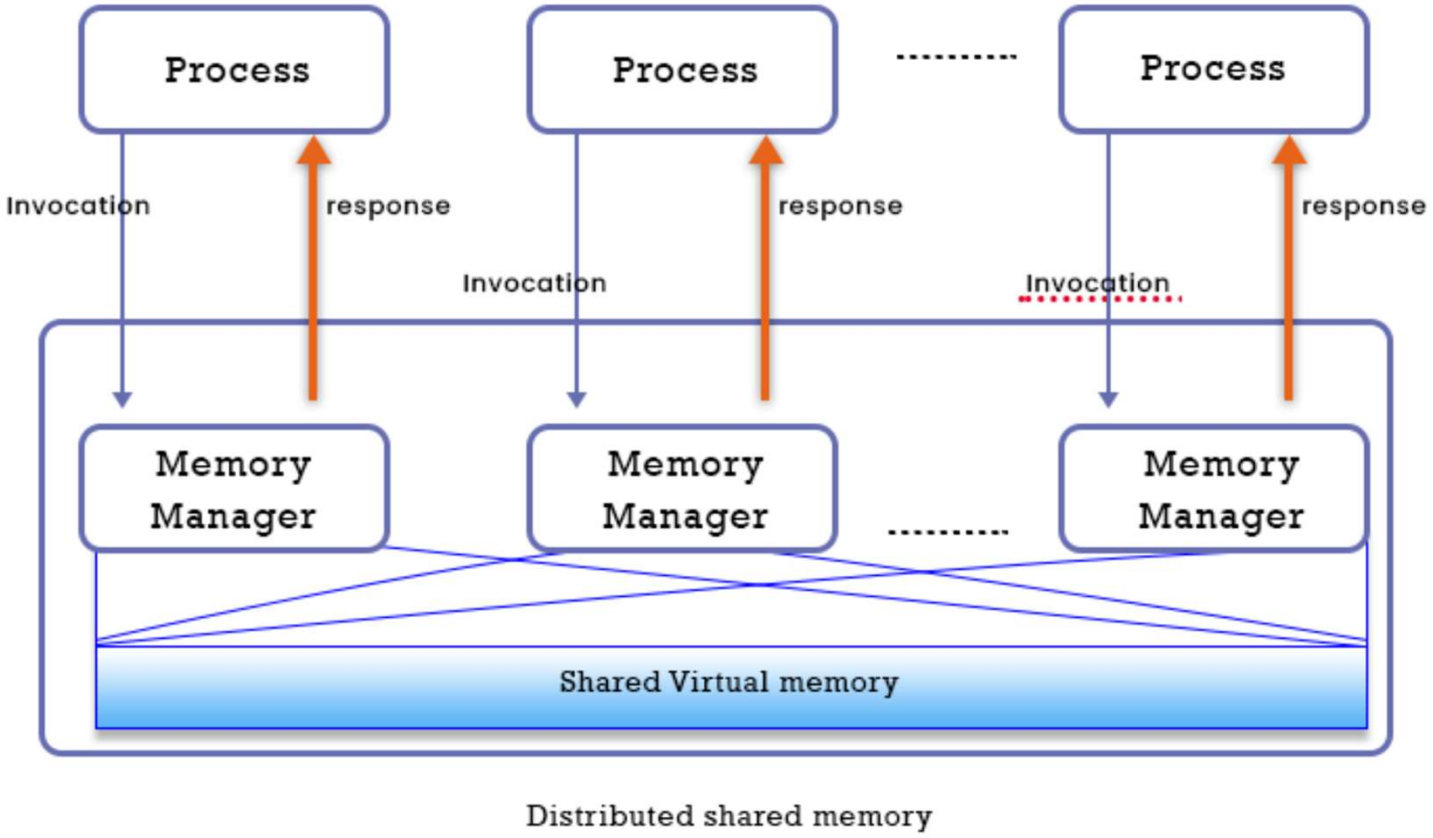
OS-Implemented DSM

- DSM system is implemented at the operating system level.
- The DSM system is responsible for managing distributed memory across multiple nodes in the system.
- To application developers, the DSM system appears as a single, unified shared memory space.
- Developers can write their applications using standard shared memory programming paradigms and APIs without needing to explicitly manage distributed memory or communication between nodes.
- The DSM system handles the underlying complexities of distributed memory management transparently, abstracting them away from the developers.

Library or Language-Level Software DSM

- DSM system is implemented as a library or language extension that developers can use in their applications.
- Unlike OS-implemented DSM, software DSM libraries provide a higher-level interface for managing distributed memory.
- Developers need to use specific APIs or language constructs provided by the DSM library to explicitly manage distributed memory and coordinate access between nodes.
- Software DSM libraries often require developers to write their applications in a specific programming paradigm.

Software based DSM



Distributed shared memory

Implementing DSM in software

- There are three ways of implementing a software distributed share memory:

1. **Page based approach** using the system virtual memory;

   1. The DSM system intercepts memory access requests and dynamically manages the mapping of virtual memory pages to physical memory locations distributed across multiple nodes.

   2. In this scenario, the DSM functionality is deeply integrated in to the operating system's memory management infrastructure, allowing processes to access shared memory transparently as if it were part of the system's virtual memory.

Implementation DSM (1)

1. **Shared variable approach** using some routines to access shared variables;
   1. DSM functionality is provided by external libraries that developers link with their applications.
   2. These libraries manage the coordination and synchronization of shared memory access.

2. Object based approach ideally accessing shared data through object oriented discipline.

1. DSM libraries provide support for encapsulating shared data within objects.
2. DSM functionality is provided by libraries that offer object-oriented abstractions for managing shared memory.

Advantages of DSM

- Scales well with a large number of nodes.
  - Unlimited number of nodes can be used. Unlike multiprocessor systems where main memory is accessed via a common bus, thus limiting the size of the multiprocessor system.

- Message passing is hidden –
  - Hide data movement and provide a simpler abstraction for sharing data.
  - Programmers don't need to worry about memory transfers between machines like when using the message passing model.

- Allows the passing of complex structures by reference, simplifying algorithm development for distributed applications.

- Takes advantage of "locality of reference" by moving the entire page containing the data referenced rather than just the piece of data.

Advantages of DSM (1)

- Cheaper to build than multiprocessor systems.
- Ideas can be implemented using normal hardware and do not require anything complex to connect the shared memory to the processors.
- Larger memory sizes are available to programs, by combining all physical memory of all nodes. This large memory will not incur disk latency due to swapping like in traditional distributed systems.
- Unlimited number of nodes can be used. Unlike multiprocessor systems where main memory is accessed via a common bus, thus limiting the size of the multiprocessor system.
- Programs written for shared memory multiprocessors can be run on DSM systems,

Disadvantage of DSM

Few disadvantage of DSM are given below:

1. Generally slower to access than non-distributed share memory.
2. Must provide additional protection against simultaneous accesses to shared data.
3. May incur a performance penalty.
4. Little programmer control over actual messages being generated.
5. Programmers need to understand consistency model, to write correct programs
6. DSM implementation uses asynchronous message passing and hence cannot be more efficient than message passing implementations.